Sarabo — Viva: Fundamental প্রশ্ন

Feature নয়, technology আর concept নিয়ে প্রশ্ন। প্রতিটা উত্তর ছোট রাখা হয়েছে, যাতে মুখে বলা যায়। Project-এর কথাগুলো আসল code মিলিয়ে লেখা (৩০ অক্টোবর ২০২৬)। Feature আর code location-এর জন্য আলাদা ফাইল: `Sarabo_Viva_QnA.pdf`।

> **কীভাবে পড়বে:** ★ চিহ্নিত প্রশ্নগুলো আগে পড়ো, এগুলো আসার সম্ভাবনা সবচেয়ে বেশি। উত্তরটা ঢেকে নিজের ভাষায় বলার চেষ্টা করো, মুখস্থ নয়। উত্তরের ধরন: **সংজ্ঞা → আমার project-এ কোথায় → একটা সীমাবদ্ধতা**। না জানলে বলো "এটা ভালো করে জানি না"; বানিয়ে বললে পরের প্রশ্নেই ধরা পড়ে।

সূচি: ১. AI ব্যবহার · ২. Tech stack · ৩. JWT ও Firebase · ৪. JavaScript · ৫. Node.js · ৬. Express.js · ৭. HTTP ও REST · ৮. React · ৯. Client-side বাকি tools · ১০. Database · ১১. System design · ১২. Security · ১৩. Testing ও deployment · ১৪. Project-এর দুর্বল দিক · ১৫. Rapid-fire

১. AI ব্যবহার নিয়ে প্রশ্ন

স্যারকে সৎভাবে বলবে, এটা ভালো সিদ্ধান্ত। তবে নিচের উত্তরগুলো নমুনা মাত্র: **তোমার ক্ষেত্রে যেটুকু সত্য শুধু সেটুকুই বলবে**, নিজের ভাষায়। এরপর স্যার যাচাই করবেন তুমি বোঝো কি না, তাই বাকি অংশগুলোই আসল প্রস্তুতি।

1. ★ এই project-এ AI কতটা ব্যবহার করেছ?

> সরাসরি বলো: code লেখায় অনেকটাই AI-এর সাহায্য নিয়েছি। Project-এর idea, কোন feature থাকবে, কোন নিয়ম চলবে, এগুলো আমি ঠিক করেছি, চালিয়ে পরীক্ষা করেছি, আর ভুল পেলে ঠিক করিয়েছি। শতাংশ বানিয়ে বলবে না।

2. ★ AI যদি code লেখে, তাহলে তুমি কী শিখলে?

> পুরো system কীভাবে একসাথে কাজ করে: request কীভাবে চলে, auth কীভাবে হয়, data কোথায় থাকে, payment কীভাবে confirm হয়। প্রতিটা line নিজে হাতে লিখিনি, কিন্তু design আর flow বুঝিয়ে বলতে পারি। আপনি যেকোনো অংশ খুলে জিজ্ঞেস করতে পারেন।

3. ★ AI ভুল করলে কীভাবে ধরলে?

> Automated test, browser-এ নিজে চালিয়ে দেখা, আর CI। উত্তরে **তোমার নিজের দেখা একটা বাস্তব ঘটনা** বলো, যেটা সত্যি ঘটেছে। আগে থেকে একটা ঠিক করে রাখো।

4. এখন AI ছাড়া একটা ছোট CRUD API বানাতে পারবে?

> পারলে হ্যাঁ বলো, আর viva-র আগে একবার করে দেখো: Express route, MongoDB insert/find, একটা React form। না পারলে সৎভাবে বলো কোন অংশে আটকাবে। স্যার সামনে করতে বলতে পারেন।

5. তোমার project-এও তো AI feature আছে। সেটা আর code লেখায় AI, দুটো কি এক?

> না। Project-এর ভেতরে Google Gemini চলে (request form draft, support chat, report summary)। আর development-এ AI ছিল coding সহকারী। এই দুটো আলাদা করে বলবে।

২. Tech stack: কোনটা কেন

কাজ	ব্যবহার করেছি	কেন	বিকল্প
UI	React 19	Component, বড় ecosystem	Vue, Angular
Build tool	Vite 7	দ্রুত dev server, সহজ config	Webpack

Routing	React Router 7	Nested layout, lazy route	Next.js routing
Server state	TanStack Query	Cache, refetch, invalidate	useEffect + useState, Redux
Styling	Tailwind CSS 4, Radix	দ্রুত, consistent, accessible UI	Bootstrap, MUI
API	Node.js + Express 5	Client-server দুদিকেই JavaScript	Django, Spring, Laravel
Database	MongoDB (Atlas), native driver	Nested document, transaction আছে	PostgreSQL, Mongoose
Auth	Firebase Authentication	Password, Google login, email verify তৈরি	নিজের JWT, Auth0
Payment	Stripe	Checkout, webhook, refund, test mode	SSLCommerz, bKash
File storage	Supabase Storage (private)	Free tier, signed URL	Cloudinary, S3
Email	Nodemailer + Gmail	Free, প্রায় ৫০০ email/দিন	SendGrid
PDF receipt	PDFKit	Server-এ PDF তৈরি	Puppeteer
Map	Leaflet + OpenStreetMap	Free, key লাগে না	Google Maps
AI	Google Gemini	Free tier, Banglish বোঝে	OpenAI
Hosting	Vercel	Free, সহজ deploy, cron	Render, VPS
Test	নিজের test suite, Playwright	Real HTTP + real DB, browser flow	Jest, Cypress

6. ★ MERN stack কী? তোমারটা কি MERN?

> MongoDB, Express, React, Node। হ্যাঁ, আমারটা MERN, সাথে Firebase Auth, Stripe, Supabase Storage আর Gemini।

7. ★ Frontend, backend আর database কেন আলাদা রেখেছ?

> Three-tier architecture: UI, business logic, data আলাদা। সব নিয়ম server-এ থাকে কারণ client-এর code user বদলাতে পারে। আলাদা deploy করা যায়, আর পরে mobile app হলে একই API চলবে।

8. তোমার project কোন architecture pattern মানে?

> Layered, MVC-র মতো: `routes` → `middleware` → `controllers` → `services` / `models` → MongoDB। View অংশটা React। পুরো API এক deployment-এ, তাই এটা monolith।

৩. Authentication: JWT ও Firebase

9. ★ JWT কী? এর ভেতরে কী থাকে?

> JSON Web Token: `header.payload.signature`। Header-এ algorithm, payload-এ claim (email, expiry), signature দিয়ে প্রমাণ হয় token বদলানো হয়নি। Server token store না করেই যাচাই করতে পারে।

10. ★ JWT কি encrypted? Payload-এ password রাখা যায়?

> না, সাধারণ JWT শুধু signed, encrypted নয়। Payload base64url, যে কেউ decode করে পড়তে পারে। তাই গোপন কিছু রাখা যাবে না। Signature শুধু পরিবর্তন ধরে।

11. ★ Firebase ব্যবহার করলে, তাহলে কি JWT ব্যবহার করোনি?

> করেছি। Firebase ID token নিজেই একটা JWT, মেয়াদ ১ ঘণ্টা। পার্থক্য হলো token বানায় Firebase, আমার server নয়। আমি নিজে `jwt.sign()` করি না; server `admin.auth().verifyIdToken()` দিয়ে যাচাই করে।

12. ★ নিজের JWT না বানিয়ে Firebase কেন?

> নিজে করলে password hashing, email verification, password reset, Google login, token refresh, key management সব লিখতে ও সুরক্ষিত রাখতে হতো। Firebase-এ এগুলো তৈরি আর free। Password আমার database-এ থাকেই না, তাই ফাঁস হওয়ারও ভয় নেই। ক্ষতি: Google-এর service-এর উপর নির্ভরতা।

13. ★ পুরো login flow বলো।

```
Client: Firebase SDK দিয়ে sign in → ID token পায়
Client: useAxiosSecure interceptor → Authorization: Bearer <token>
Server: verifyFBToken → verifyIdToken() → req.decoded_email
Server: verifyAdmin / verifyTechnician → users collection থেকে role চেক
Server: controller কাজ করে → response
```

14. ★ Authentication আর authorization-এর পার্থক্য?

> Authentication: তুমি কে (token যাচাই, ব্যর্থ হলে 401)। Authorization: তোমার অনুমতি আছে কি না (role, ownership, ব্যর্থ হলে 403)।

15. Session-based আর token-based auth-এর পার্থক্য?

> Session: server state রাখে, browser-এ শুধু session id (সাধারণত cookie)। Token: তথ্য token-এই থাকে, server কিছু রাখে না, তাই stateless ও serverless-এ সুবিধা। Session সহজে সাথে সাথে বাতিল করা যায়, JWT যায় না।

16. ★ Logout করলে JWT কি সাথে সাথে invalid হয়?

> না। Client token ফেলে দেয়, কিন্তু কেউ কপি করে থাকলে মেয়াদ শেষ পর্যন্ত (১ ঘণ্টা পর্যন্ত) চলবে। আমার code-এ `verifyIdToken` revocation চেক করে না। দরকার হলে `checkRevoked: true` দেওয়া যায়, তবে প্রতি request-এ বাড়তি network call হবে।

17. Token-এর মেয়াদ শেষ হলে কী হয়? Access আর refresh token কী?

> Access token স্বল্পমেয়াদি, refresh token দিয়ে নতুন access token আনা হয়। Firebase SDK এটা নিজেই করে: `getIdToken()` মেয়াদ শেষের কাছাকাছি হলে refresh করে। তবু 401 এলে আমার interceptor একবার forced refresh করে retry করে, এরপরও ব্যর্থ হলে logout।

18. Token কোথায় রাখা উচিত? localStorage না cookie?

> localStorage XSS-এ পড়া যায়। httpOnly cookie JS পড়তে পারে না, কিন্তু CSRF protection লাগে। আমার ক্ষেত্রে Firebase SDK নিজে browser storage-এ (default IndexedDB) রাখে, আর token যায় header-এ। Cookie ব্যবহার করি না, তাই CSRF-এর ঝুঁকি কম, কিন্তু XSS থেকে সাবধান থাকতে হয়।

19. Server প্রতিবার token যাচাই করতে Google-কে call করে?

> না। Firebase token RS256-এ signed: Google private key দিয়ে sign করে, আর Admin SDK Google-এর public key (cache করা) দিয়ে নিজেই যাচাই করে। Signature, expiry, audience, issuer সব চেক হয়।

20. HS256 আর RS256-এর পার্থক্য?

> HS256 symmetric: একই secret দিয়ে sign ও verify। RS256 asymmetric: private key দিয়ে sign, public key দিয়ে verify, তাই যাচাইকারী token বানাতে পারে না।

21. ★ Role JWT-তে রাখোনি কেন? Database-এ কেন?

> Firebase custom claim-এ রাখা যেত, তাহলে প্রতিবার DB read লাগত না। কিন্তু token-এ থাকলে role বদলালে পুরনো token মেয়াদ পর্যন্ত পুরনো role বহন করে। DB-তে রাখায় পরিবর্তন সাথে সাথে কার্যকর। ক্ষতি: admin/technician route-এ একটা বাড়তি query।

22. Client-এর `AdminRoute` থাকলে server-এ `verifyAdmin` কেন?

> Client route guard শুধু UI লুকায়; Postman দিয়ে সরাসরি API call করা যায়। আসল নিরাপত্তা server-এ।

23. Firebase-এর web `apiKey` client-এ দেখা যায়। সমস্যা না?

> না। এটা project চেনার জন্য, গোপন key নয়। গোপন হলো server-এর service account key (`FB_SERVICE_KEY`), যেটা শুধু server-এর env-এ থাকে। Admin SDK সেটা দিয়ে চলে।

24. Email verification কীভাবে enforce করেছ?

> Decoded token-এর `email_verified` দেখে `verifyEmailVerified` middleware; না হলে 403। Client-এর পাঠানো কোনো flag বিশ্বাস করি না।

৪. JavaScript

25. ★ `var`, `let`, `const`-এর পার্থক্য?

> `var` function-scoped, hoisted হয়ে `undefined` পায়। `let`/`const` block-scoped, declare-এর আগে ব্যবহার করলে error (Temporal Dead Zone)। `const` reassign করা যায় না, তবে object-এর ভেতরের মান বদলানো যায়।

26. ★ Closure কী? একটা উদাহরণ দাও।

> Function তার তৈরি হওয়ার জায়গার variable মনে রাখে, বাইরের function শেষ হলেও। উদাহরণ: `useAxiosSecure`-এর interceptor; আর React-এর "stale closure" সমস্যা। আমার interceptor পুরনো `user` ধরে রাখছিল বলে `auth.currentUser` পড়ে।

27. ★ Event loop কী?

> JavaScript single-threaded। Call stack খালি হলে আগে সব microtask (Promise), তারপর একটা macrotask (setTimeout, I/O callback) চলে। এভাবে thread আটকে না রেখে async কাজ হয়।

28. এই code-এর output কী?

```
console.log(1); setTimeout(() => console.log(2), 0);
Promise.resolve().then(() => console.log(3)); console.log(4);
```

> `1 4 3 2`। আগে sync কোড, তারপর microtask, তারপর timer।

29. ★ Promise আর async/await কী?

> Promise ভবিষ্যৎ ফলের object: pending, fulfilled, rejected। `async/await` Promise-এর উপরেই সহজ syntax; `await` ঐ function থামায়, পুরো thread নয়। Error ধরতে `try/catch`। আগে callback nesting-এর সমস্যা ছিল (callback hell)।

30. `Promise.all` আর `Promise.allSettled`-এর পার্থক্য?

> `all` একটা fail করলেই পুরোটা reject। `allSettled` সবার ফল দেয়, সফল বা ব্যর্থ। পরস্পর নির্ভরশীল না হলে parallel চালালে সময় বাঁচে।

31. `==` আর `===`?

> `==` type বদলে তুলনা করে (`"5" == 5` true), `===` type-সহ (false)। সবসময় `===` ব্যবহার করা ভালো।

32. `null` আর `undefined`?

> `undefined`: মান দেওয়া হয়নি। `null`: ইচ্ছা করে "কিছু নেই"। `AuthProvider`-এ দুটোর আলাদা অর্থ আছে: `undefined` মানে এখনও জানা নেই, `null` মানে signed out।

33. Arrow function আর সাধারণ function-এর পার্থক্য?

> Arrow-র নিজের `this` ও `arguments` নেই, বাইরেরটা নেয়; `new` দিয়ে চলে না। সাধারণ function-এ `this` নির্ভর করে কীভাবে call হলো তার উপর।

34. Hoisting কী?

> Declaration আগে memory-তে নেওয়া হয়, তাই function declaration লেখার আগেই call করা যায়। `var` পায় `undefined`, `let/const` TDZ-তে থাকে।

35. Prototype কী?

> JS-এর inheritance পদ্ধতি। Object-এ property না পেলে prototype chain ধরে খোঁজে। `class` এর উপরেই syntax। আমার `RepairRequestModel` একটা class।

36. Spread, destructuring, optional chaining কী?

> `{...a}` কপি/merge। `const { user } = useAuth()` বের করে আনা। `error.response?.status` null হলে crash না করে `undefined` দেয়।

37. Shallow copy আর deep copy?

> Shallow শুধু উপরের স্তর কপি করে, ভেতরের object একই reference। Deep সব স্তর (`structuredClone`)। React state বদলাতে এটা জরুরি, কারণ React reference তুলনা করে।

38. `map`, `filter`, `reduce`, `forEach`?

> `map` রূপান্তর করে নতুন array, `filter` বাছাই, `reduce` এক মানে আনে (যেমন total), `forEach` শুধু loop, কিছু return করে না।

39. CommonJS আর ES Module?

> `require/module.exports` বনাম `import/export`। আমার server CommonJS, client ES Module (`"type": "module"`)। ESM static, তাই tree-shaking ভালো হয়।

40. Debounce কী?

> পরপর আসা call-এর শেষটা কিছুক্ষণ পরে একবার চালানো; যেমন search-এ প্রতি অক্ষরে API call না করা।

৫. Node.js

41. ★ Node.js কী? এটা কি language বা framework?

> কোনোটাই না। Node হলো JavaScript runtime: V8 engine + libuv, যা browser-এর বাইরে JS চালায় এবং file, network access দেয়।

42. ★ Node single-threaded হয়ে অনেক request কীভাবে সামলায়?

> Non-blocking I/O। DB বা network call শুরু করে অপেক্ষা না করে পরের request ধরে; কাজ শেষ হলে event loop callback চালায়। JS এক thread-এ চলে, তবে libuv-এর thread pool আর OS I/O-র কাজ করে।

43. তাহলে Node কোন কাজে খারাপ?

> CPU-ভারী কাজ (video encoding, বড় হিসাব), কারণ event loop আটকে যায়। সমাধান worker threads বা আলাদা service। আমার project I/O-ভারী, তাই মানানসই।

44. Blocking আর non-blocking code-এর উদাহরণ?

> `fs.readFileSync` শেষ না হওয়া পর্যন্ত সব আটকে রাখে; `await fs.promises.readFile` রাখে না।

45. `package.json` আর `package-lock.json`-এর পার্থক্য?

> `package.json`-এ চাহিদা ও script; lock file-এ সব dependency-র হুবহু version, যাতে সব জায়গায় একই install হয়। `dependencies` production-এ লাগে, `devDependencies` শুধু development-এ।

46. `^5.1.0` মানে কী?

> Semantic versioning: major.minor.patch। `^` মানে একই major-এ নতুন minor/patch চলবে (`<6.0.0`)।

47. Environment variable কেন? `.env` commit করো?

> Secret আর environment অনুযায়ী config code-এর বাইরে রাখতে। `.env` commit হয় না, `.env.example`-এ শুধু নামগুলো থাকে। Production-এ Vercel-এর settings-এ।

48. Buffer আর Stream কী?

> Buffer: raw bytes। Stream: data টুকরো টুকরো পড়া, পুরোটা memory-তে না এনে। Stripe webhook-এ raw Buffer লাগে signature যাচাইয়ের জন্য।

৬. Express.js

49. ★ Express কী? Node থাকতে Express কেন?

> Node-এর উপর ছোট web framework। শুধু Node-এর `http` দিয়ে URL parsing, routing, body parsing নিজে করতে হতো; Express-এ routing আর middleware তৈরি।

50. ★ Route কী? Parts গুলো বলো।

```
app.post('/repair-requests/:id/quote/decision',
  verifyFBToken, ensureDatabaseReady, verifyEmailVerified,
  (req, res) => quoteController.decideQuote(req, res));
```

> HTTP method + path + middleware chain + handler। `:id` হলো route parameter। বাম থেকে ডানে চলে; একটা response দিয়ে দিলে পরেরগুলো চলে না।

51. ★ Middleware কী? `next()` কী করে?

> Request আর response-এর মাঝের function, `(req, res, next)`। `next()` পরের ধাপে পাঠায়, না ডাকলে আর response না দিলে request ঝুলে থাকে। আমার উদাহরণ: `requestId`, `verifyFBToken`, `verifyAdmin`, rate limiter।

52. `req.params`, `req.query`, `req.body`?

> Params: `/requests/:id` এর মান। Query: `?status=open`। Body: POST/PATCH-এর JSON, `express.json()` parse করে দেয়।

53. `app.use` আর `app.get`?

> `app.use` সব method-এ, path prefix মিলিয়ে চলে। `app.get` শুধু GET-এ, পুরো path মিলিয়ে।

54. ★ Middleware-এর order কেন গুরুত্বপূর্ণ? উদাহরণ দাও।

> Express যে ক্রমে register হয় সে ক্রমে চালায়। Stripe webhook route `express.json()`-এর আগে, কারণ signature যাচাইয়ে আসল raw bytes লাগে; একবার parse হলে আসল bytes আর পাওয়া যায় না। Auth আগে, DB check পরে, যাতে ভুল token দ্রুত 401 পায়।

55. Error-handling middleware কেমন হয়?

> চারটা argument: `(err, req, res, next)`, সব route-এর পরে register করতে হয়। Express 5-এ async handler-এর rejected Promise নিজে থেকে error handler-এ যায়; Express 4-এ হাতে `next(err)` দিতে হতো। আমার `index.js`-এ CORS error-এর জন্য একটা আছে।

56. `express.Router()` কী? তুমি ব্যবহার করেছ?

> Route গুলো আলাদা ফাইলে ভাগ করার mini app। আমি একটু ভিন্নভাবে করেছি: প্রতিটা `routes/*.js` একটা function export করে, `userRoutes(app, controllers)`, যেটা সরাসরি `app`-এ register করে। উদ্দেশ্য একই।

57. `trust proxy` কেন দিয়েছ?

> Vercel-এর পেছনে থাকায় আসল client IP থাকে `X-Forwarded-For` header-এ। Rate limiter-এর জন্য আসল IP লাগে। `1` মানে একটা proxy hop বিশ্বাস করি; `true` দিলে IP জাল করা সহজ হতো।

58. `express.json()` -এর limit কত? কেন বদলেছ?

> Default 100 kB। শুধু `/ai/request-draft` -এ 2.5 MB, কারণ সেখানে base64 ছবি আসে। Limit থাকলে বিশাল body পাঠিয়ে server-কে ক্ষতি করা কঠিন।

৭. HTTP ও REST

59. ★ REST কী? এর নিয়মগুলো?

> API ডিজাইনের ধরন: URL resource বোঝায়, HTTP method কাজ বোঝায়, stateless, client-server আলাদা। আমার API মূলত RESTful; `/accept` -এর মতো action path পুরোপুরি REST-সম্মত নয়, তবে পড়তে স্পষ্ট।

60. ★ GET, POST, PUT, PATCH, DELETE-এর পার্থক্য? PUT আর PATCH?

> পড়া, তৈরি, পুরোটা বদল, আংশিক বদল, মোছা। GET, PUT, DELETE idempotent; POST নয়।

61. ★ Status code: 200, 201, 400, 401, 403, 404, 409, 429, 500, 503?

> OK; created; ভুল input; login নেই; অনুমতি নেই; নেই; state conflict; অতিরিক্ত request; server error; service সাময়িক অনুপলব্ধ। 401 আর 403-এর পার্থক্য নিশ্চিত বলতে পারবে।

62. ★ Browser-এ URL লিখে Enter চাপলে কী হয়?

> DNS থেকে IP → TCP connection → TLS handshake → HTTP request → server response → HTML parse, CSS/JS load → render। আমার site-এ Vercel `index.html` দেয়, তারপর React চালু হয়ে API call করে।

63. ★ CORS কী? এটা কি server-কে রক্ষা করে?

> Browser-এর নিয়ম: অন্য origin-এর response পড়তে server-এর অনুমতি লাগে। Client আর API আলাদা domain, তাই `config/cors.js` -এ allow-list। এটা শুধু browser-এর জন্য; Postman বা curl আটকায় না। আসল সুরক্ষা token।

64. Preflight request কী?

> `Authorization` header বা JSON body থাকলে browser আগে `OPTIONS` পাঠিয়ে জেনে নেয় অনুমতি আছে কি না, তারপর আসল request পাঠায়।

65. HTTP আর HTTPS?

> HTTPS = HTTP + TLS: encryption, integrity, server-এর পরিচয় প্রমাণ। Bearer token পাঠানোর জন্য অপরিহার্য।

66. Stateless মানে কী? কেন ভালো?

> Server আগের request মনে রাখে না, সব তথ্য প্রতিটা request-এ। যেকোনো instance যেকোনো request সামলাতে পারে, তাই scale সহজ।

67. REST আর GraphQL? REST কেন?

> GraphQL-এ client ঠিক করে কোন field লাগবে, একটা endpoint। REST সহজ, HTTP cache চলে, আর আমার দরকার মিটিয়েছে।

৮. React

68. ★ React কী? Library না framework?

> UI বানানোর library। Component দিয়ে declarative UI। Routing বা data fetching নিজে দেয় না, তাই আলাদা library নিয়েছি।

69. ★ Virtual DOM ও reconciliation কী?

> Render-এ UI-এর হালকা JS tree তৈরি হয়, আগের tree-র সাথে diff করে শুধু বদলানো অংশ আসল DOM-এ যায়। আসল DOM update তুলনামূলক খরচের।

70. ★ Props আর state-এর পার্থক্য?

> Props বাইরে থেকে আসে, read-only। State component-এর নিজের, বদলালে re-render হয়।

71. ★ `useState` আর `useEffect` কী? Dependency array-র তিন রূপ?

> State রাখা, আর render-এর পরে side effect চালানো। Array নেই: প্রতি render-এ। `[]` : mount-এ একবার। `[x]` : `x` বদলালে। Cleanup function unmount-এ বা পরের effect-এর আগে চলে, যেমন `onAuthStateChanged` unsubscribe আর interceptor eject।

72. ★ Hook-এর নিয়ম কী? Custom hook কী?

> শুধু top level-এ আর component/custom hook থেকে call; condition বা loop-এ নয়, কারণ React call-এর ক্রম দিয়ে state মেলায়। Custom hook হলো `use` নামের function যা logic আবার ব্যবহার করে: `useAuth`, `useRole`, `useAxiosSecure`।

73. List-এ `key` কেন লাগে? Index দিলে সমস্যা কী?

> Key দিয়ে React চেনে কোন item কোনটা। Index দিলে order বদলালে ভুল item-এ state যুক্ত হতে পারে; তাই stable id ভালো।

74. `useMemo`, `useCallback`, `useRef`?

> `useMemo` হিসাবের ফল মনে রাখে, `useCallback` function-এর reference। `useRef` এমন মান রাখে যা বদলালে re-render হয় না, বা DOM element ধরে; `AuthProvider`-এ আগের uid রাখতে ব্যবহার করেছি।

75. ★ Context API কী? Redux কেন নাওনি?

> Prop drilling ছাড়া data পৌঁছানো। আমি auth আর theme-এ Context ব্যবহার করেছি। বেশিরভাগ data server থেকে আসে, সেটা TanStack Query সামলায়। তাই Redux-এর আলাদা global store লাগেনি। Context-এর value বদলালে সব consumer re-render হয়, তাই ঘন ঘন বদলানো data-র জন্য ভালো নয়।

76. ★ TanStack Query কী করে? `useEffect`-এ fetch করলে কী সমস্যা?

> `useEffect`-এ loading, error, cache, race condition সব হাতে লিখতে হয়। Query-তে `queryKey` দিয়ে cache, আর mutation-এর পরে `invalidateQueries`।

77. Controlled আর uncontrolled component?

> Controlled: input-এর মান React state-এ। Uncontrolled: DOM-এই, ref দিয়ে পড়া। react-hook-form মূলত uncontrolled, তাই re-render কম।

78. SPA কী? CSR, SSR, SSG-র পার্থক্য?

> SPA: এক HTML, route বদলায় browser-এ। CSR browser-এ render, SSR প্রতি request-এ server-এ, SSG build-এর সময়। আমারটা CSR; dashboard-ভিত্তিক app বলে ঠিক আছে, তবে SEO দুর্বল।

79. ★ Protected route কীভাবে কাজ করে? Refresh দিলে login পেজে যায় না কেন?

> `PrivateRoute` user না থাকলে `<Navigate to="/login">` দেয়। Firebase session ফেরাতে সময় লাগে, তাই `loading` true থাকা পর্যন্ত Loading দেখায়। এটা না থাকলে refresh-এ ভুল করে login-এ পাঠাত।

80. Lazy loading আর code splitting কী?

> `lazy(() => import(...))` দিয়ে page-এর JS আলাদা chunk হয়, দরকারে load হয়। `Suspense` ততক্ষণ fallback দেখায়। প্রথম load দ্রুত হয়।

81. `StrictMode`-এ effect দুবার চলে কেন?

> শুধু development-এ, cleanup ভুল আছে কি না ধরার জন্য। Production-এ একবার।

82. JSX কী? Browser কি সরাসরি বোঝে?

> JS-এর ভেতরে HTML-এর মতো syntax। Browser বোঝে না; build tool JS function call-এ রূপান্তর করে।

83. Re-render কখন হয়? কমানোর উপায়?

> নিজের state, parent-এর re-render, বা context বদলালে। State নিচে নামানো, `React.memo`, `useMemo`। Code-এ `memo` ব্যবহার করিনি, সমস্যা মেপে দরকার পড়লে করতাম।

৯. Client-side বাকি tools

84. Vite কী? Create React App থেকে ভালো কেন?

> Dev-এ browser-এর native ES module দিয়ে file on demand দেয়, তাই শুরু ও HMR দ্রুত। Production build bundle করে minify করে। CRA এখন deprecated।

85. Tailwind কী? Inline CSS থেকে পার্থক্য?

> Utility class দিয়ে style। Inline-এ hover, media query, design token হয় না; Tailwind-এ হয়, আর শুধু ব্যবহৃত class build-এ থাকে।

86. Axios কেন, `fetch` কেন নয়? Interceptor কী?

> Axios-এ interceptor, 4xx/5xx-এ reject, আর auto JSON। Interceptor প্রতিটা request/response-এর মাঝে চলে: আমি token লাগাই আর 401 সামলাই।

87. `VITE_` prefix কেন? সেখানে secret রাখা যায়?

> শুধু `VITE_` variable browser bundle-এ যায়। তাই secret রাখা যাবে না, build করা JS-এ যে কেউ দেখতে পারে।

88. Vercel-এ client-এ rewrite কেন লাগল?

> `/dashboard` সরাসরি খুললে ঐ নামে কোনো file নেই, 404 হতো। `vercel.json` সব path `index.html`-এ পাঠায়, তারপর React Router সামলায়।

১০. Database

89. ★ MongoDB কেন? SQL কেন নয়?

> Request-এ nested data আছে (product, quote, inspection, payment), আর ভিন্ন device-এর field ভিন্ন, তাই document সহজে মেলে। JSON JS-এর সাথে স্বাভাবিক। Free Atlas আছে।

90. ★ Payment আর booking আছে, তাহলে SQL বেশি ঠিক ছিল না?

> সৎ উত্তর: PostgreSQL-ও খুব ভালো হতো, relation আর constraint শক্তিশালী। MongoDB-তে এই নিশ্চয়তা আমি multi-document transaction আর unique index দিয়ে নিয়েছি। Trade-off: consistency-র দায় বেশিটা application code-এ।

91. ★ SQL আর NoSQL-এর পার্থক্য?

> Table/row, fixed schema, JOIN বনাম document, flexible schema, embedding। SQL সাধারণত vertical scale, NoSQL সহজে horizontal (sharding)।

92. ★ Embedding আর referencing: কখন কোনটা?

> একসাথে পড়া হয় ও সংখ্যায় কম হলে embed: quote, inspection → `repair_requests`-এ। অনেক হলে বা আলাদাভাবে খোঁজা লাগলে reference: offer → `job_applications`। Document-এর সীমা 16 MB।

93. ★ তোমার database design বলো।

> 21টা collection। মূল: `users`, `technicians`, `repair_requests`, `service_definitions`, `job_applications`, `payments`, `technician_withdrawals`, `warranty_claims`, `notifications`, `tracking_events`। Relationship: এক customer-এর অনেক request; এক request-এর অনেক offer; এক request-এ একজন booked technician।

94. ★ Index কী? কীভাবে কাজ করে? ক্ষতি কী?

> সাজানো data structure (B-tree), তাই পুরো collection scan লাগে না। ক্ষতি: write ধীর হয়, জায়গা লাগে। আমার `config/database.js`-এ 44টা `createIndex`।

95. Compound index-এ field-এর ক্রম কেন গুরুত্বপূর্ণ?

> Leftmost prefix: `{technicianId, status, createdAt}` শুধু `status` দিয়ে খোঁজায় ভালো কাজ করে না। সাধারণ নিয়ম: Equality, Sort, Range।

96. Unique, partial, TTL index?

> Unique: duplicate আটকায় (`trackingId`)। Partial: কিছু document-এ (একটাই খোলা withdrawal)। TTL: সময় হলে নিজে মোছে (`email_log`)। Code-এ bug থাকলেও DB আটকায়।

97. ★ ACID আর transaction?

> Atomicity সব বা কিছুই না; Consistency নিয়ম ঠিক থাকে; Isolation একে অন্যকে নষ্ট করে না; Durability commit টিকে থাকে। Booking-এ `session.withTransaction`। MongoDB-তে replica set লাগে।

98. ★ Race condition কী? দুজন একসাথে একই job নিলে?

> একসাথে চলা কাজের ফল সময়ের উপর নির্ভর করলে। আমি condition-সহ update করি: `{ _id, deliveryStatus: 'pending-pickup', technicianId: { $exists: false } }`, আর transaction-এ। দ্বিতীয়জনের update মেলে না। Unique index শেষ সুরক্ষা।

99. Mongoose কেন নয়? তাহলে schema কোথায়?

> Native driver ব্যবহার করেছি; transaction, aggregation-এ সরাসরি নিয়ন্ত্রণ। ক্ষতি: Mongoose-এর মতো তৈরি schema validation নেই। Validation নিজে লিখেছি `utils/`-এ (যেমন `repairRequestV2.js`), আর `models/`-এ class।

100. Normalization কী? MongoDB-তে denormalize করো?

> Duplicate কমাতে data ভাগ করা (1NF, 2NF, 3NF)। হ্যাঁ, করি: price snapshot request-এ কপি করা, যাতে পরে দাম বদলালেও পুরনো request ঠিক থাকে।

101. Aggregation pipeline কী? `$lookup`?

> ধাপে ধাপে data process: `$match`, `$group`, `$sort`। Admin report-এ ব্যবহার করেছি। `$lookup` মানে SQL-এর left join-এর মতো।

102. `ObjectId` কী?

> 12-byte id: timestamp (4), random (5), counter (3)। Client বা server-এই তৈরি হয়, তাই distributed-এ সংঘর্ষ প্রায় হয় না।

103. Replica set আর sharding?

> Replica set: একই data-র কপি, primary-তে write, primary গেলে election; availability দেয়। Sharding: data ভাগ করে রাখা; capacity বাড়ায়। আমি sharding ব্যবহার করিনি, দরকার নেই।

104. Pagination কীভাবে করে? Offset আর cursor?

> Offset: `skip/limit`, সহজ কিন্তু গভীরে ধীর। Cursor: শেষ `_id`-এর পর থেকে, দ্রুত। আমি offset ব্যবহার করেছি (যেমন `models/Notification.js`, `models/RepairRequest.js`), কারণ data কম আর page নম্বর দেখানো সহজ। Data অনেক বড় হলে cursor দেব।

১১. System design

105. ★ পুরো system-এর architecture বলো।

```
Browser (React, Vercel CDN)
  → HTTPS → Express API (Vercel serverless, Mumbai bom1)
      → MongoDB Atlas (data)   → Firebase (auth যাচাই)
      → Stripe (payment, webhook ফিরে আসে)
      → Supabase (ছবি)   → Gemini (AI)   → Gmail (email)
```

106. ★ Monolith আর microservices? তুমি কোনটা, কেন?

> Monolith: এক deployment। Microservices: ছোট আলাদা service, আলাদা scale, কিন্তু network, deployment, debugging জটিল। আমি একা, data কম, তাই monolith, কিন্তু folder-এ module করে ভাগ করা।

107. ★ Serverless কী? সুবিধা-অসুবিধা?

> Server-এর চিন্তা platform-এর, request এলে function চলে। সুবিধা: free tier, auto scale। অসুবিধা: cold start, memory ভাগ হয় না, দীর্ঘ connection চলে না, সময়সীমা।

108. ★ Serverless-এ MongoDB connection কীভাবে সামলাও?

> প্রতি request-এ নতুন connection দিলে connection limit শেষ হবে। `connectionPromise` module-এ cache করি; warm instance-এ একটাই connection। Fail হলে cache মুছে পরের request-এ retry।

109. ★ Horizontal আর vertical scaling?

> Vertical: বড় machine। Horizontal: বেশি machine + load balancer। Stateless API হওয়ায় আমার API horizontal scale করতে পারে; Vercel নিজেই instance বাড়ায়।

110. ১০,০০০ user হলে কী বদলাবে?

> আগে পরিমাপ করে bottleneck খুঁজব। সম্ভাব্য: cursor pagination, slow query-তে index, Redis cache, বড় Atlas tier, email/AI কাজ queue-তে, আর paid email service।

111. Caching কোথায় আছে?

> Client: TanStack Query। CDN: hashed asset-এ `Cache-Control: immutable`। Server-side Redis নেই, দরকার পড়েনি। কঠিন অংশ হলো invalidation।

112. Rate limiting কী? তোমার implementation?

> নির্দিষ্ট সময়ে request সীমিত করা। Public tracking: 20/মিনিট, in-memory। AI support: MongoDB-তে shared counter, কারণ serverless instance-এ memory ভাগ হয় না। সেই কারণে tracking-এর limit প্রতি instance-এ আলাদা, এটা একটা দুর্বলতা।

113. Real-time-এর জন্য WebSocket কেন নয়?

> Serverless-এ দীর্ঘ connection সহজ নয়। Job board আর admin attention প্রতি ৬০ সেকেন্ডে poll করে। সত্যিকারের real-time লাগলে আলাদা service লাগবে। Polling, long polling, WebSocket, SSE-র পার্থক্য জেনে রাখো।

114. Idempotency কেন distributed system-এ জরুরি?

> Network ব্যর্থ হলে retry হয়, Stripe webhook একই event একাধিকবার পাঠাতে পারে। `payments.sessionId` unique আর `idempotencyKey` দিয়ে একবারই কাজ হয়।

115. CAP theorem কী?

> Network partition হলে consistency আর availability-র মধ্যে একটা বেছে নিতে হয়। MongoDB-কে সাধারণত CP ধরা হয়।

116. Single point of failure কোথায়?

> Firebase, Atlas বা Stripe বন্ধ হলে সেই অংশ থামে। DB বন্ধ হলে `ensureDatabaseReady` 503 দেয়; email ব্যর্থ হলে মূল কাজ থামে না।

117. Cron কী? `0 12 * * *` মানে?

> Minute hour day month weekday। প্রতিদিন 12:00 UTC, মানে বাংলাদেশে সন্ধ্যা ৬টা। Vercel Cron `/cron/pickup-reminders` call করে।

১২. Security

118. ★ XSS কী? React কীভাবে রক্ষা করে?

> Page-এ ক্ষতিকর script ঢোকানো। React `{value}` escape করে। আমার client code-এ `dangerouslySetInnerHTML` নেই।

119. ★ CSRF কী? তোমার project-এ ঝুঁকি আছে?

> Browser নিজে cookie পাঠায় বলে অন্য site থেকে request করানো। আমি cookie নয়, `Authorization` header ব্যবহার করি, তাই এই আক্রমণ সাধারণত কাজ করে না।

120. ★ NoSQL injection কী?

> `{"email": {"$ne": null}}` এর মতো object পাঠিয়ে query বদলানো। উপায়: type চেক, input validate। ঝুঁকি থাকে JSON body-তে, তাই server validation জরুরি।

121. ★ IDOR কী? কীভাবে আটকিয়েছ?

> অন্যের id দিয়ে তার data দেখা। Login-ই যথেষ্ট নয়, প্রতিটা controller-এ দেখি user owner, assigned technician, না admin। Email নেওয়া হয় verified token থেকে, body থেকে নয়।

122. Password কীভাবে store হওয়া উচিত? Hash আর encryption?

> Salt-সহ ধীর hash (bcrypt, scrypt, Argon2)। Hash একমুখী, encryption ফেরানো যায়। Password Firebase-এ থাকে, আমার DB-তে নয়।

123. Client থেকে আসা দাম বিশ্বাস করো?

> না। পরিমাণ server DB থেকে নেয়। Client-এর validation শুধু UX।

124. Stripe webhook অন্য কেউ call করলে?

> `constructEvent` raw body আর `STRIPE_WEBHOOK_SECRET` দিয়ে signature যাচাই করে; না মিললে reject।

125. Private ছবি কীভাবে সুরক্ষিত?

> Bucket private, স্বল্পমেয়াদি signed URL। Link ফাঁস হলেও অল্প সময়ের জন্য।

126. Tracking page-এ login নেই। নিরাপদ?

> Code অনুমান করা কঠিন: 128-bit random (`crypto`), আর rate limit। ক্ষতি: code যার কাছে সে দেখতে পারে, তাই সংবেদনশীল তথ্য সেখানে দেখানো হয় না।

১৩. Testing ও deployment

127. ★ Unit, integration, e2e-র পার্থক্য? তোমার কোনগুলো?

> Unit: একটা function। Integration: কয়েকটা অংশ একসাথে। E2E: user-এর মতো browser থেকে। আমার server test আসল HTTP আর MongoDB-তে (integration), Playwright e2e। Client-এ unit test নেই।

128. CI/CD কী? তোমার আছে?

> CI: প্রতি push-এ test (GitHub Actions, আলাদা MongoDB container-এ)। Deploy স্বয়ংক্রিয় CI-তে নেই, হাতে করা হয়। তাই পুরো CD নয়।

129. Git: merge আর rebase? `fetch` আর `pull`?

> Merge history রাখে, rebase সরলরেখা করে। `fetch` শুধু আনে, `pull` = fetch + merge।

130. Development আর production data কীভাবে আলাদা?

> আলাদা database name (`MONGO_DB_NAME`) আর আলাদা env। CI-কে Atlas-এর credential দেওয়া হয় না।

১৪. Project-এর দুর্বল দিক (এগুলো নিজেই বলতে পারলে ভালো)

"তোমার project-এর দুর্বলতা কী?" প্রশ্ন আসে প্রায়ই। নিজে জানলে বোঝা যায় তুমি বুঝে করেছ। নিচেরগুলো code থেকে পাওয়া, মিথ্যা নয়।

দুর্বলতা	কেন এমন	কীভাবে ঠিক করতাম
Logout-এর পরেও token মেয়াদ পর্যন্ত চলে	Revocation চেক করি না	সংবেদনশীল route-এ `checkRevoked`
Client-এ unit test নেই	শুধু Playwright	Vitest + Testing Library
`helmet` নেই	Security header আলাদা set করা হয়নি	`helmet` যোগ
Public tracking rate limit in-memory	Instance-ভেদে আলাদা গণনা	Support-এর মতো shared store
Real-time নেই	Serverless, polling করি	আলাদা WebSocket service
SPA-র SEO দুর্বল	Client-side rendering	Public page SSR/pre-render
Cold start	Serverless	Free uptime ping বা always-on server

১৫. Rapid-fire (এক লাইনে)

প্রশ্ন	উত্তর
V8 কী?	Google-এর JS engine, JS-কে machine code করে

npm কী?	Node-এর package manager
`typeof null` ?	`"object"`, পুরনো bug
API কী?	দুই software-এর কথা বলার নিয়ম
JSON কী?	Text-ভিত্তিক data format
CRUD?	Create, Read, Update, Delete
CDN?	User-এর কাছাকাছি server থেকে static file দেওয়া
DNS?	Domain থেকে IP
Port 443, 80?	HTTPS, HTTP
Hook কী?	`use` দিয়ে শুরু, function component-এ state/lifecycle দেয়
Prop drilling?	মাঝের component দিয়ে props নামানো
HMR?	Reload ছাড়া বদলানো module update
Tree shaking?	অব্যবহৃত code bundle থেকে বাদ
Bearer token?	যার কাছে আছে সেই ব্যবহার করতে পারে
OAuth?	অন্য service দিয়ে login/অনুমতি (Google sign-in)
Webhook?	Event হলে অন্য service আমাকে call করে
Replica set কেন লাগল?	Multi-document transaction-এর জন্য
Foreign key আছে MongoDB-তে?	না, reference রাখি, DB enforce করে না
Node version LTS কী?	দীর্ঘমেয়াদি support পাওয়া version
Vercel region কেন Mumbai?	Atlas আর বাংলাদেশের কাছাকাছি, latency কম